RISK-4-AUTO: Residually Interconnected and Superimposed Kolmogorov-Arnold Networks for Automotive Network Traffic Classification

Anurag Dutta, *Graduate Student Member, IEEE*, Sangita Roy, *Member, IEEE*, and Rajat Subhra Chakraborty, *Senior Member, IEEE*

***Abstract*—In modern automobiles, a Controller Area Network (CAN) bus facilitates communication among all electronic control units for critical safety functions, including steering, braking, and fuel injection. However, due to the lack of security features, it may be vulnerable to malicious bus traffic-based attacks that cause the automobile to malfunction. Such malicious bus traffic can be the result of either external *fabricated messages* or *direct injection* through the on-board diagnostic port, highlighting the need for an effective intrusion detection system to efficiently identify suspicious network flows and potential intrusions. This work introduces Residually Interconnected and Superimposed Kolmogorov-Arnold Networks (RISK-4-AUTO), a set of four deep neural network architectures for intrusion detection targeting in-vehicle network traffic classification. RISK-4-AUTO models, when applied on three hexadecimally identifiable sequence-based open-source datasets (collected through *direct injection* in the on-board diagnostic port), outperform six state-of-the-art vehicular network intrusion detection systems (as per their accuracies) by ≈1.0163% for all-class classification and ≈2.5535% on focused (single-class) malicious flow detection. Additionally, RISK-4-AUTO enjoys a significantly lower overhead than existing state-of-the-art models, and is suitable for real-time deployment in resource-constrained automotive environments.**

***Index Terms*—Controller area network (CAN), in-vehicle security, Kolmogorov-Arnold Network (KAN), network forensics, network traffic classification.**

I. INTRODUCTION

MODERN automobiles depend entirely on embedded electronic devices, commonly termed Electronic Control Units (ECUs) [1] for critical functions like steering, fuel injection, braking, etc. These ECUs communicate through Controller Area Network (CAN) bus systems following the ISO 11898 standard [2]. The communication over a CAN-bus follows a broadcast mechanism to ensure efficiency, reliability, and the ability to present a consistent system-state snapshot to all ECUs [3]. The CAN data frame format and the protocol for the CAN-bus communication schemes ensure excellent message integrity, and thus CAN is very well-suited for environments with inherently high electromagnetic interference and electrical noise, e.g., vehicles and industrial production environments.

However, the protocol lacks any in-built cryptographic mechanism to ensure message confidentiality and sender authenticity [3]. As a result, the communication between ECUs is vulnerable to being hijacked by any attacker, resulting in severe automobile malfunctions, sufficient to wreak havoc on the road, and posing a threat to lives and infrastructures. Malicious CAN messages capable of causing malfunction of the ECUs can be generated in two major ways [3]:

1) Through direct physical injection of fabricated CAN messages [4] on the CAN-bus via the on-board diagnostic port of the vehicle, or,
2) via fabricated messages injected to the CAN-bus by a telematics device [5] capable of wireless communication with external agents. This threat is exacerbated by the existence of wireless communication technology and standards (e.g., IEEE 802.11p [6], [7]) for vehicular communication.

A modern vehicle has an extensive attack surface with numerous and diverse attack vectors. For example, it has been recently demonstrated that Android infotainment units etc. can also serve as attack vectors [8].

To counter these threats, efficient and accurate Intrusion Detection System (IDS) needs to be deployed, which monitors the CAN-bus in real-time, and in case of any suspicious traffic, immediately quarantines (or flushes, if needed) such data frames. Efficient specialized IDS for vehicular network traffic have been proposed in several recent works. Tariq et al. combined the heuristics of Reinforcement Learning with Recurrent Neural Networks (RNN) [9], and later in another work made use of Convolutional Long Short-Term Memory (Conv-LSTM) network for classification of network traffic on a CAN-bus [10]. Similarly, Song et al. suggested using Deep Convolutional Neural Network (DCNN) [3], and Javed et al. hybridized Convolution Neural Network (CNN) with Attention-based Gated Recurrent Unit (GRU) [11]. Seo et al. added to the pool by using Generative Adversarial Network (GAN) for data augmentation, and further improving

Received 14 January 2025; revised 12 June 2025 and 5 October 2025; accepted 22 October 2025. Date of publication 24 October 2025; date of current version 29 December 2025. The associate editor coordinating the review of this article and approving it for publication was Y. Diao. *(Corresponding author: Rajat Subhra Chakraborty.)*

Anurag Dutta and Rajat Subhra Chakraborty are with the Department of Computer Science and Engineering, Indian Institute of Technology Kharagpur, Kharagpur 721302, India (e-mail: anuragdutta@ieee.org; rschakraborty@cse.iitkgp.ac.in).

Sangita Roy is with the Department of Computer Science and Engineering, Thapar Institute of Engineering and Technology, Patiala 147004, India (e-mail: sangita.roy@thapar.edu).

Digital Object Identifier 10.1109/TNSM.2025.3625404

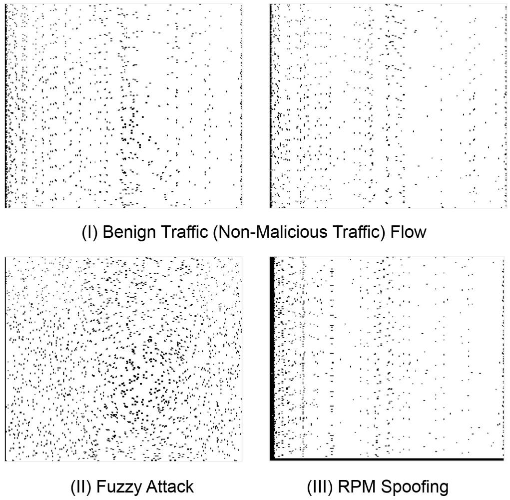


Fig. 1. Some examples of the generated two-dimensional visual representation of *spatial* and *temporal* characteristics of in-vehicle network traffic from the `HCRL-Car Hacking Dataset`. The horizontal axis represents normalized frame arrival timestamps (in seconds), and the vertical axis representing the corresponding frame sizes (in bytes).

the prediction efficiency [12]. Ashraf et al. proposed a novel architecture, LSTM-Autoencoder-IDS (LSTMIDS) [13], which to the best of our knowledge, reports the best classification performance till date.

In this paper, we introduce **Residually Interconnected and Superimposed Kolmogorov-Arnold Networks** (RISK-4-AUTO), a novel combination of four deep learning-based IDS. RISK-4-AUTO models combines *Residual Neural Network* (ResNet) [14], [15], [16] and *Kolmogorov-Arnold Network* (KAN) [17], [18], [19], for malicious in-vehicular network traffic classification, in particular those resulting from *direct injection* attacks. The RISK-4-AUTO IDS models make use of visual classification techniques to distinguish among different frames for a given network flow.

Fig. 1 exemplifies such a visual representation for three classes (one benign, two malicious) from the `HCRL-Car Hacking Dataset` (in-vehicle network traffic), with the horizontal axis representing normalized frame arrival timestamps (in seconds), and the vertical axis representing the corresponding frame sizes (in bytes). It can be observed visually from the figure that the benign flow of traffic is *densely*, *uniformly*, and more importantly *periodically* well-distributed, whereas the injected flow is either *sparsely* (due to anomalous timing characteristics) or *non-uniformly* distributed (due to sudden spikes in message size, frequency, etc). The deep learning models learn the underlying patterns from these visual representations in a supervised manner. Later, the trained models are applied to distinguish between different classes of in-vehicle network traffic flows. Fig. 2 depicts the three phases.

All of the four proposed models under RISK-4-AUTO achieved superior performance compared to the state-of-the-art (S-O-T-A), when applied on the HCRL network traffic datasets [20], [21], [22]. For example, the best classification accuracy achieved by RISK-4-AUTO was 99.76% for distinguishing five (four malicious and one benign) different types of network flows of the `HCRL-Car Hacking Dataset`, compared to the previous best-reported accuracy of 99.01%. Further, we evaluated the models (on each of the three datasets) for their ability to distinguish between injected and normal flow of traffic from every malicious flow category (focused detection), where for each of those attacks corresponding to the three HCRL datasets, one or more RISK-4-AUTO models achieved a perfect (i.e., 100% accurate) classification.

The rest of the paper is organized as follows: Section II presents some notable works from the existing literature that aligns the research problem, and our contributions. Section III discusses the preliminaries necessary for proposing the RISK-4-AUTO Intrusion Detection Systems for automotive network classification. Section V presents the supervised framework, the adopted *mini-FlowPic* algorithm for histogram generation, the three HCRL datasets and the four proposed deep neural network architectures. Section VI describes the experimental setup, and the experimental results to evaluate the effectiveness of the RISK-4-AUTO IDS models. Finally, Section VII concludes the research.

II. Related Works and Our Contributions

This section reviews some works from the existing literature on the topic under consideration, some of which have been mentioned already in Section I.

Tariq et al. combined reinforcement learning-based adaptive threshold adjustment with rule-based heuristics (e.g., message rate, ID validity) with a Recurrent Neural Network to learn temporal attack signatures [9]. They also suggested using a reward-driven fine-tuning to update anomaly thresholds online, thus improving the adaptability, and generalizability to new (unidentified) traffic patterns. One major limitation of this technique is the complexity of reward shaping, as poorly designed reward functions can lead to increased false positive rates. Also, from a practical point of view, training RNNs for automotive intrusion detection typically requires substantial volumes of labeled data, which makes both the turn-around time, and developmental cost high. In a follow-up work [10], they proposed CAN-Transfer, where pre-trained Convolution-LSTM (Conv-LSTM) was used on CAN dataset. Herein, the Convolutional layers extracted spatial feature patterns from CAN message representations, and the LSTM captured temporal dependencies. Although it achieved the-then state-of-the-art accuracy in intrusion detection, the accuracy depends on the volume of data available for fine-tuning the modules; in case of scarce data availability, performance degrades.

Song et al. suggested using Deep Convolutional Neural Network (DCNN) [3] for mapping CAN frames to 2D images (time vs. ID axes), eliminating the need for manual feature engineering and ensuring low false negative and therefore lower overall error rates. This technique's effectiveness was significantly influenced by the volume and quality of labeled training data available for training the network. Further, the underlying network architecture under this DCNN involves large depth of convolutional layers, which increases the inference time, compared to shallow-networks. Javed et al. proposed CANIntelliIDS [11], where they hybridized Convolution Neural Network (CNN) with Attention-based Gated Recurrent Unit (AGRU). The

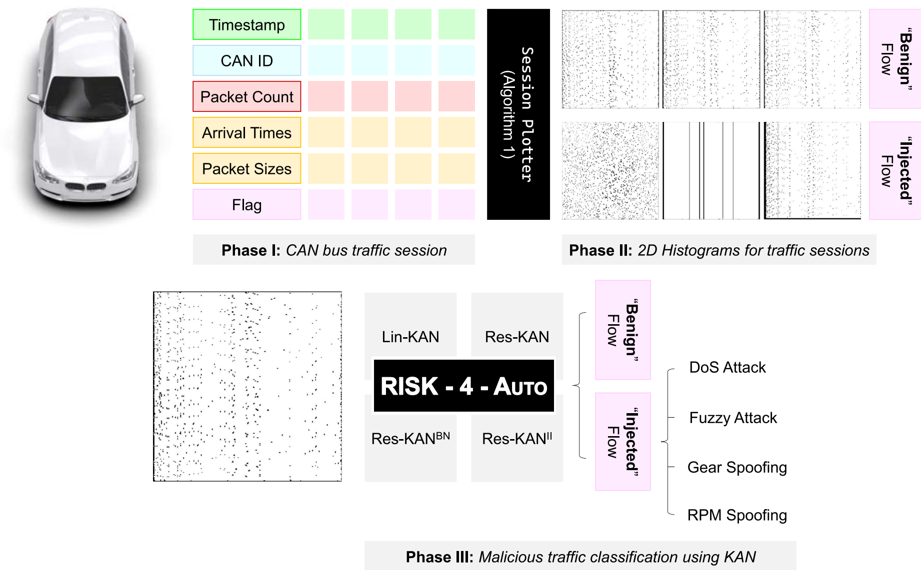


Fig. 2. Pictorial demonstration for the workflow of the RISK-4-AUTO framework. The methodology has three phases: (I) capturing the CAN bus traffic session from the ECUs of any automobile; (II) employing a "Session Plotter" (Algorithm 1) for generation of two-dimensional histograms, and annotating them based on the "Flag" field present in the session header, and, (III) finally, the malicious traffic classification is performed in a supervised manner using *Kolmogorov-Arnold Network*.

CNN backbone performed spatial feature extraction, and Attention-augmented GRU focused on class-specific salient features. They further reported ≈10% improvement on mixed-attack benchmarks, compared to prior models. Despite achieving state-of-the-art accuracy, a major shortcoming of CANIntelliIDS was its memory overhead during inference due to attention mechanisms. Seo et al. proposed GIDS [12], where the CAN traffic data was augmented using spectral-normalized GAN. It further increased the model robustness to noise and often generalizable to newer samples, without explicit training. However, GAN based augmentation may (in some instances) generate unrealistic samples (as the generation is unconstrained), which cause the training to be unstable. Additionally, the synthetic samples make the detection module vulnerable to rote-learning, by memorizing the GAN generated patterns. Yang et al. proposed MTH-IDS [23] (a three-tier detection architecture) combining signature-based filtering at the edge, anomaly detection in fog, and global correlation in the cloud. It hybridizes a lightweight rule-based check with a deep-autoencoder and random-forest as an ensemble model balancing speed and detection depth. It is also vulnerable to communication latency due to multi-tier coordination.

Araujo-Filho et al. [24] implement a machine learning-based Intrusion Prevention System (IPS), using isolation forest algorithms on CAN frame payload data to achieve real-time attack detection and prevention within timing constraints on resource-constrained hardware. Ashraf proposed LSTM-Autoencoder-IDS (LSTMIDS) [13], trains an LSTM-autoencoder on benign CAN sequences (only), and flags the frames whose reconstruction error exceeds a threshold as anomalies. Till date, it achieves state-of-the-art accuracy with ≈99% accuracy on select benchmark datasets.

A. Our Contributions

The proposed RISK-4-AUTO models outperform the limitations of the existing state-of-the-art, as they do not require a very large volume of data, in contrast to [9], [10]. They map the time-series to histograms (refer to Algorithm 1), increasing information gain per data-instance. Besides, unlike state-of-the-art models which have very-deep networks for classification [3], RISK-4-AUTO models have limited residually-connected convolutional layers (one-two; refer to Fig. 3) and thus offers faster inference, with no memory or computational overhead arising from additional model complexity (like attention mechanism [11], coordination-latency [23] etc). **Additionally, RISK-4-AUTO achieves ≈1.0163% improvement for *all-class classification*, and ≈2.5535% on focused (*single-class*) malicious flow detection, besides relatively low inference time, in comparison to lightweight detection systems from the literature like [10], [24]. This is also one of most important requirement for implementation of any model on a resource-constrained environment.**

III. BACKGROUND: VISUAL REPRESENTATION OF NETWORK TRAFFIC, RESNET AND KAN

A. FlowPic for Visual Representation of Network Packets

"*FlowPic*" [25], [26] was one of the early contributions towards *visual representation-based* techniques for network traffic identification. Shapira et al. in their contribution devised a methodology to convert network traffic time series (continuous) data to their (discrete) visual representation, as histograms. By normalizing both arrival times and sizes into a fixed $[0, \#\ \texttt{MTU} - 1]$ range and binning into an $(\#\ \texttt{MTU} \times \#\ \texttt{MTU})$ grid, *FlowPic* yields a grayscale image capturing spatiotemporal traffic patterns. In their seminal work, Shapira et al. considered $\texttt{MTU} = 1500$, which was updated to 64, 128, 256 respectively in their follow-up work, *mini-FlowPic* [27]. Inspired by the performance boost, we inherit *mini-FlowPic* with dimension 64×64 in Section V.

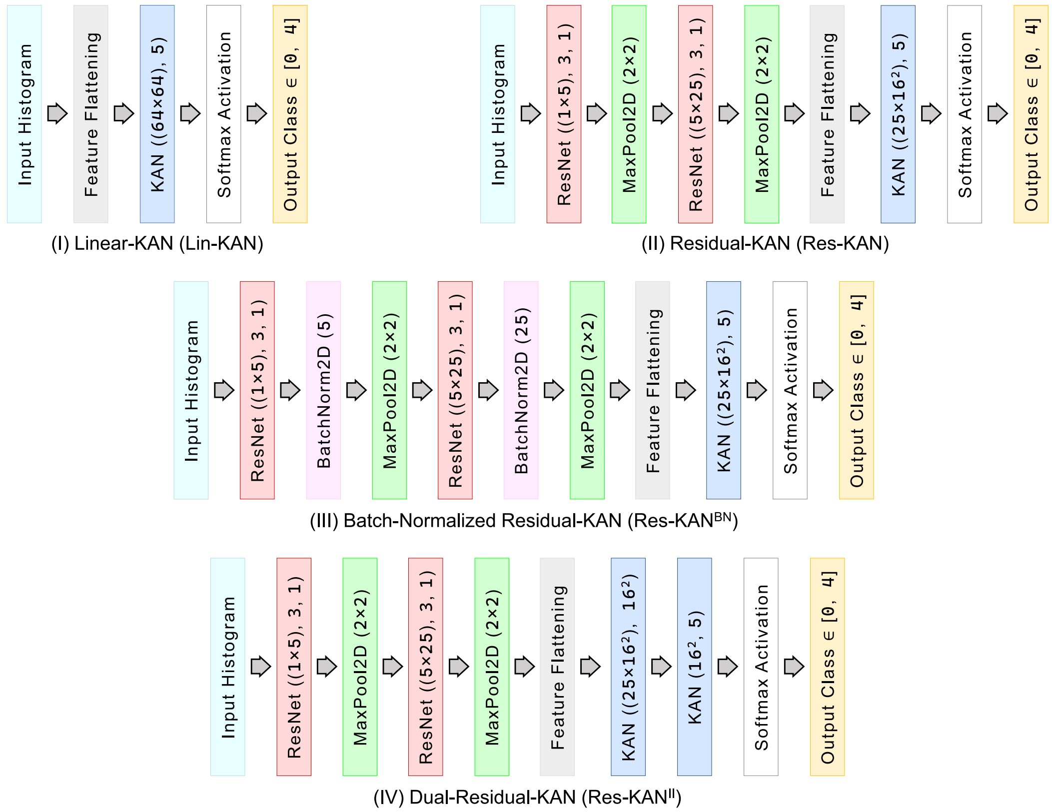


Fig. 3. Proposed architectures of RISK-4-AUTO Intrusion Detection Systems for malicious automotive traffic classification through visualization.

B. Residual Neural Network (ResNet)

AlexNet [28] was the first CNN-based architecture that got mass attention, particularly after it's astonishing performance at the ImageNet 2012 competition [29], [30], [31]. Subsequently, every other model that aimed to outperform AlexNet relied on using more layers of computation as a measure to overcome the margin of error, but this came up with a trade-off. Adding more layers introduced the *Vanishing Gradient* (gradient tends to zero) [32], and *Exploding Gradient* (gradient tends to infinity) [33] problems in the deep neural networks. Residual Blocks came to the rescue, by enabling the deeper neural networks to train on a given dataset with lower experimental error. Residual Blocks introduced *Skip Connection* that connects activations from non-adjacent layers by skipping layers in between. This enables the network to skip any layer by regularization if that layer contributes to degrading performance of the model. Mathematically, the Residual Block operation is represented as in eqn. (1):

$$\mathbf{x} \mapsto a[\mathbf{x}] + \mathbf{x} \tag{1}$$

where $a[\mathbf{x}]$ refers to the activation from any neural layer with input feature $\mathbf{x}$.

Since our proposed RISK-4-AUTO models solely depend on learning patterns from *FlowPic* images generated for automotive network traffic flow, the vanishing gradient problem may adversely affect the precision of the IDS. Hence, we have made Residual Blocks an integral part of the proposed RISK-4-AUTO models.

C. Kolmogorov-Arnold Network (KAN)

Kolmogorov Arnold Network (KAN) was introduced in [17] to overcome the limitations of the existing Multi-layer Perceptrons (MLPs). MLPs decide the numerical values of the node weights using *fixed activation functions*, whereas KAN updates the weights using *trainable activation functions*. Inheriting from the "*Universal Approximation Theorem*", for MLPs any function $y = f(x)$ can be approximated as in eqn. (2)

$$\mathbf{y} = \sum a[\boldsymbol{\Omega} \cdot \mathbf{x} + \boldsymbol{\beta}] \tag{2}$$

where $a[\,\cdot\,]$ is the **fixed** activation function (e.g., `ReLU`, `tanH`, etc.), and $(\omega_{i,j} \in \boldsymbol{\Omega}, \beta_j \in \boldsymbol{\beta})$ are the **model weights**, and **bias** respectively. Contrary to that, inheriting from the "*Kolmogorov-Arnold Representation Theorem*", the approximation (for KAN) can be made as in eqn. (3)

$$\mathbf{y} = \sum \boldsymbol{\Phi}\Big[\sum \boldsymbol{\phi}[\mathbf{x}]\Big] \tag{3}$$

where $\phi_{i,j} \in \boldsymbol{\phi}$ are the real-valued functions (**trainable, specifically B-Splines**), and $\Phi_\ell \in \boldsymbol{\Phi}$ is the mapping (composition) function of ℓ^{th} layer. Further, the real-valued functions are defined as:

$$\phi_{i,j}[x_i] = \mu_{i,j}^{(b)} \cdot \left(\frac{x_i}{1 + e^{-x_i}}\right) + \mu_{i,j}^{(s)} \cdot \sum_{k=1}^{\mathcal{K}} \lambda_{i,j,k} \cdot \mathcal{B}_k[x_i] \tag{4}$$

and the composition functions as:

$$\Phi_\ell = \begin{bmatrix} \phi_{\ell,1,1} & \phi_{\ell,1,2} & \cdots & \phi_{\ell,1,\mathrm{L}} \\ \phi_{\ell,2,1} & \phi_{\ell,2,2} & \cdots & \phi_{\ell,2,\mathrm{L}} \\ \vdots & \vdots & \ddots & \vdots \\ \phi_{\ell,\mathrm{L},1} & \phi_{\ell,\mathrm{L},2} & \cdots & \phi_{\ell,\mathrm{L},\mathrm{L}} \end{bmatrix}; \quad \phi_{\ell,i,j} \equiv \phi_{i,j} \tag{5}$$

Algorithm 1 Histogram Generation Algorithm

Require: n, $\boldsymbol{\mathcal{T}}$, and $\boldsymbol{\Pi}$ ▷ $\boldsymbol{\mathcal{T}}$, $\boldsymbol{\Pi}$ are n-dim vectors
Ensure: $\mathcal{H}_{i,i} \in \boldsymbol{\mathcal{H}}$ ▷ $\boldsymbol{\mathcal{H}}$ is the 2-D Histogram
function SESSIONPLOTTER(n, $\boldsymbol{\mathcal{T}}$, $\boldsymbol{\Pi}$)
  Initialize vectors $\tilde{\boldsymbol{\mathcal{T}}}$, $\tilde{\boldsymbol{\Pi}}$ with `NULL` entries.
  for $i = 0$ to $n - -1$ **do**
    $\tilde{\mathcal{T}}_i \leftarrow \left(\frac{\mathcal{T}_i - \mathcal{T}_0}{\mathcal{T}_{n-1} - \mathcal{T}_0}\right) \cdot \#\ \texttt{MTU}$ ▷ Normalization of packet arrival times
    $\tilde{\Pi}_i \leftarrow \left(\frac{\Pi_i - \Pi_0}{\Pi_{n-1} - \Pi_0}\right) \cdot \#\ \texttt{MTU}$ ▷ Normalization of packet sizes
  end for
  Initialize $\boldsymbol{\mathcal{H}}_{(\#\ \texttt{MTU} \times \#\ \texttt{MTU})} \leftarrow \mathbf{0}_{(\#\ \texttt{MTU} \times \#\ \texttt{MTU})}$
  for $j = 0$ to $n - -1$ **do**
    $\mathcal{H}_{\lfloor \tilde{\Pi}_j \rfloor, \lfloor \tilde{\mathcal{T}}_j \rfloor} \leftarrow \mathcal{H}_{\lfloor \tilde{\Pi}_j \rfloor, \lfloor \tilde{\mathcal{T}}_j \rfloor} + 1$ ▷ Increment
  end for
  return $\boldsymbol{\mathcal{H}}$
end function

where, $(\mu^{(b)}_{i,j}, \mu^{(s)}_{i,j})$ are the weights for the "*biological*" term (`Swish`), and "*spline*" term ($\lambda_{i,j,k}$ being the spline coefficients).

IV. Proposed Solution

This section, describes the supervised framework, the adopted *mini-FlowPic* algorithm for histogram generation and the four classifier model architectures constituting RISK-4-AUTO.

A. The Supervised Framework

This framework (refer to Fig. 2) considers sequential data stream (from the CAN bus) consisting of standard network parameters in the form, {Timestamp (in UTC), CAN message Identifier (in Hexadecimal), Number of packets (n), n packet arrival time stamps (normalized form: $\boldsymbol{\mathcal{T}}$), n tuples of packet sizes (normalized form: $\boldsymbol{\Pi}$, in bytes)} and a "Flag"}.

The "Flag" field represents the status of the message under consideration, i.e., either the message is injected on the CAN bus, or it is evolving originally. The time-series corresponding to aforementioned set of five (excluding "Flag") **features** are mapped to form a two-dimensional histogram (as per Algorithm 1), which is then further used for the classification (both *multi-class* and *binary*) using supervised learning algorithms. Algorithm 1 constructs a 2D histogram that captures the joint distribution of (normalized) packet arrival times and their sizes within a network session. First, both packet arrival times ($\boldsymbol{\mathcal{T}}$) and sizes ($\boldsymbol{\Pi}$) are normalized to a fixed resolution (as constrained by the definition of `# MTU`). These normalized values are then used to index into the 2D histogram grid $\boldsymbol{\mathcal{H}}$, incrementing the count at the corresponding location for each packet, which enables visualization of *spatial* and *temporal* characteristics of the network traffic.

B. RISK-4-AUTO: *Residually Interconnected and Superimposed Kolmogorov Arnold Networks*

This subsection describes the four classifier model architectures constituting RISK-4-AUTO, as depicted pictorially in Fig. 3.

1) *Linear KAN* (LIN-KAN) is the most simple, and lightweight classifier of all the RISK-4-AUTO IDS models. It takes a visual histogram of dimension (64 $\times$ 64) as input, then rearranges (through a `Flatten` Layer) the entire feature set from the two-dimensional histogram into a single one-dimensional array consisting of 4096 features, and inputs it to the KAN layer in batches of 32 features each. The KAN layer further learns from the underlying features, such that for any input feature set, it maps the set (through `Softmax` Activation Layer) to any one of the network traffic classes (five in our case).
2) *Residual KAN* (RES-KAN) is the next model in the cluster. It has an incremental architecture on the LIN-KAN model, wherein before channelizing the feature vectors to the KAN layers, they are passed through two consecutive sequences of Residual Layers, and `MaxPool2D` (with kernel size 2 $\times$ 2) for better extraction of the spatio-temporal feature. The first network maps between $n_{\text{input}} = 1$ and $n_{\text{output}} = 5$, while the second network maps between $n_{\text{input}} = 5$ and $n_{\text{output}} = 25$. Afterwards, the model retraces the computational path as followed by LIN-KAN.
3) *Batch-Normalized Residual KAN* (RES-KAN$^{\text{BN}}$) is similar to the RES-KAN model, the only difference being an additional learnable `BatchNorm2D` layer after each of the Residual Layers. The `BatchNorm2D` layers scale and shift the normalized activations from the respective predecessors, improve the flow of features and gradients between the corresponding layers, and counter the internal Covariate Shift.
4) *Dual-Residual-KAN* (RES-KAN$^{\text{II}}$), which is the best-performing model in the cluster. It again has an incremental architecture on the RES-KAN model, with two KAN layers for feature extraction and mapping. This way, the extraction is smoother than its predecessors, thus retaining and transmitting deep features, which in turn makes the classification more accurate.

The rationale behind the choice of specific model parameters is as follows; Firstly, the output histograms generated by Algorithm 1 are of dimension 64 $\times$ 64. Consequently, the input histograms to the RISK-4-AUTO model retain the same dimensions. These histograms are grayscale, with gray values normalized to the range [0, 255], and, being spread across a single channel, correspond to a total of 4096 features. Secondly, the first residual network layer in each of the RISK-4-AUTO models is configured with $n_{\text{input}} = 1$ and $n_{\text{output}} = 5$, while the subsequent layer uses $n_{\text{input}} = 5$ and $n_{\text{output}} = 25$. As the input histogram is represented as a two-dimensional grayscale image, where the intensity at each pixel corresponds to the frequency of specific features, this setup effectively captures unimodal information ($n_{\text{input}} = 1$). The choice of $n_{\text{output}} = 5$ ensures that the input is expanded into multiple feature maps, facilitating the extraction of spatial relationships in the histogram. This design aligns conceptually with (a maximum of) five network traffic classes, as is the upper bound for most of the automotive network traffic datasets. But if needed, the dimension can be expanded based on requirements. The subsequent layer, configured with $n_{\text{input}} = 5$ and

$n_{\text{output}} = 25$, maintains a scaling ratio of five, consistent with the preceding layer. Finally, the use of `MaxPool2D` with a kernel size of 2×2 allows for downscaling the features while retaining most of the relevant information. A kernel size of two is a commonly adopted choice for smaller image samples.

Additionally, the KAN layer had the following hyperparameters: `grid-size` $= 10$ (complexity of the network), `spline-order` $= 3$, that determines the order of the trainable activation functions – *B-Splines* (here degree-three polynomials), `noise-factor` $= 10^{-2}$ (for regularization, and to prevent overfitting). These values were initialized randomly, and 10% data was left out for validation during model training to determine the effective parameter values. Subsequently, these parameter values were updated through cross-validation.

V. Evaluation Methodology

This section presents the datasets and the methodology (experimental setup) used for validating the proposed RISK-4-AUTO models.

A. Dataset

In this research, for experimentation, three open-source CAN traffic based datasets made available by the "*Hacking and Countermeasure Research Lab* (HCRL)" were chosen. The datasets include Car-Hacking Dataset (HCRL-CHD) [20], CAN with flexible data rate Dataset (HCRL-CAN-FD) [21] and Bus-type topology CAN (HCRL-B-CAN) [22] Dataset were chosen. These datasets (as reported in the data collection methodology by HCRL) were collected through direct-injection in the on-board diagnostic port. The reason behind the choice of these datasets is the large number of data points available with these datasets. Besides, existing literature mostly considers these datasets (as a whole, or in sets), which enabled us to compare state-of-the-art with the proposed framework. Refer to Table I, II and III for the dataset details. Each of these three datasets is a hexadecimally identifiable sequence-based dataset consisting six main attributes, as discussed in Section IV-A. The three datasets have the following characteristics:

1) The Car-Hacking Dataset [20] covers five types of CAN traffic of which one is for benign flow, and the remaining four are: (I) DoS attack, (II) Fuzzy attack, (III) Gear spoofing attack, and (IV) RPM spoofing attack.
2) CAN with flexible data rate Dataset [21] covers three types of CAN traffic: (I) Flooding attack, (II) Fuzzy attack, and (III) Malfunction.
3) Finally, the Bus-type topology CAN Dataset [22] covers three types of CAN traffic of which one is for benign flow, and the remaining two are: (I) DDoS attack, and (II) Fuzzy attack.

B. Experimental Setup

The RISK-4-AUTO deep learning models were implemented in Python using the `PyTorch (v. 2.4)` framework, and were evaluated on three open-source datasets as discussed in Section V-A. All experiments were carried out on a system with 16 GB main memory, `Intel Core i7-1065G7 @1.30 GHz` processor and an `NVIDIA GeForce MX330` Graphics Processing Unit (GPU) with 2 GiB in-built memory. Existing literature mostly performs an *all-class classification*, but to the best of our knowledge, no other published work has reported *intra-class* detection for each class of these attacks. We have made the codes and processed data (2D histograms, some selected instances) open-sourced at [34] for reproduction.

TABLE I
AN OVERVIEW OF THE HCRL-CAR HACKING DATASET

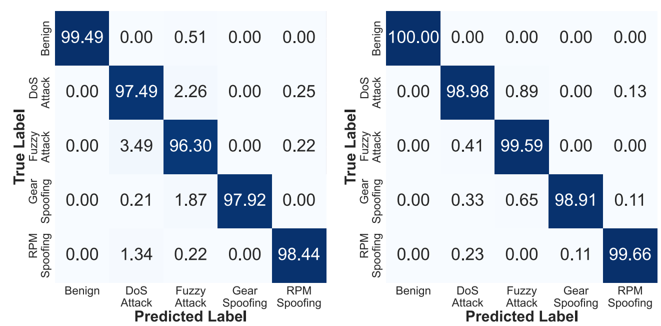
Flow Category	# Injected Messages	# Normal Messages
DoS Attack	5,87,521	30,78,250
Fuzzy Attack	4,91,847	33,47,013
Gear Spoofing	5,97,252	38,45,890
RPM Spoofing	6,54,897	39,66,805
Benign	9,88,987	

TABLE II
AN OVERVIEW OF THE HCRL CAN WITH FLEXIBLE DATA RATE DATASET

Flow Category	# Injected Messages	# Normal Messages
Flooding Attack	9,88,241	17,96,070
Fuzzy Attack	5,06,506	18,95,836
Malfunction	1,35,726	17,98,223

TABLE III
AN OVERVIEW OF THE HCRL B-CAN DATASET

Flow Category	# Injected Messages	# Normal Messages
DDoS Attack	4,00,000	25,10,262
Fuzzy Attack	1,00,000	25,10,262
Benign	25,10,262	



(a) LSTM Autoencoder based IDS (state-of-the-art model)
(b) Dual Residual KAN IDS (best-of-the-proposed model)

Fig. 4. Confusion Matrices for *all-class-classification* of network traffic on the HCRL-CHD.

VI. Experimental Results

This section describes in detail, the obtained results (Section VI-A) from the experiments mentioned in Section V-B, and also performs an ablation study (Section VI-B) on different components of the RISK-4-AUTO model, and their influence (overall) on the intrusion detection efficiency.

TABLE IV
COMPARISON OF PERFORMANCE BETWEEN DIFFERENT IDS MODELS FOR *All-Class-Classification* (MULTI-CLASS) OF NETWORK TRAFFIC ON THE HCRL-CAR HACKING DATASET

Metrics	State-of-the-art Intrusion Detection Systems (SOTA-IDS)						RISK-4-AUTO Models (proposed)			
	LSTMIDS	**GIDS**	**CANintelliIDS**	**DCNN**	**CANTransfer**	**RNN+Heuristics**	**LIN-KAN**	**RES-KAN**	**RES-KAN^{BN}**	**RES-KAN^{II}**
Accuracy	0.9901	0.9872	0.9374	0.8847	0.8544	0.6039	0.9883	0.9966	0.9917	**0.9976**
Precision	0.9902	0.9798	0.9369	0.8797	0.8593	0.7025	0.9905	0.9969	0.9924	**0.9978**
Recall	0.9902	0.9846	0.9391	0.8897	0.8495	0.5053	0.9886	0.9967	0.9925	**0.9978**
F1-Score	0.9902	0.9824	0.9379	0.8847	0.8524	0.5878	0.9893	0.9968	0.9925	**0.9978**
TTTr (seconds)	2016.48	986.73	1019.57	827.37	904.61	1175.02	620.13	1134.82	1232.27	2438.17
TTTe (seconds)	9.07	8.21	8.62	7.04	8.47	9.09	8.46	9.61	10.87	12.01

TABLE V
COMPARISON OF PERFORMANCE BETWEEN DIFFERENT IDS MODELS FOR *All-Class-Classification* (MULTI-CLASS) OF NETWORK TRAFFIC ON THE HCRL-CAN WITH FLEXIBLE DATA RATE DATASET

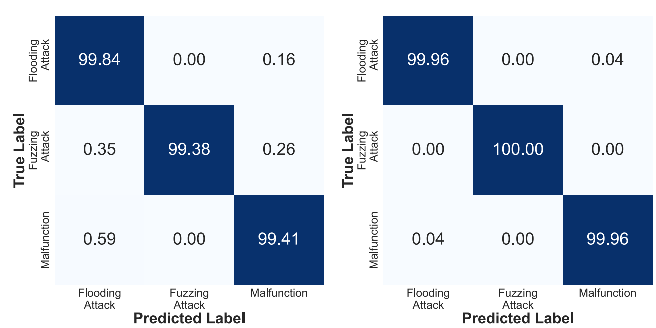
Metrics	State-of-the-art Intrusion Detection Systems (SOTA-IDS)						RISK-4-AUTO Models (proposed)			
	LSTMIDS	**GIDS**	**CANintelliIDS**	**DCNN**	**CANTransfer**	**RNN+Heuristics**	**LIN-KAN**	**RES-KAN**	**RES-KAN^{BN}**	**RES-KAN^{II}**
Accuracy	0.9928	0.9947	0.9884	0.9335	0.8994	0.7826	0.9928	0.9986	0.9992	**0.9992**
Precision	0.9925	0.9949	0.9883	0.9352	0.8986	0.7723	0.9925	0.9986	0.9991	**0.9992**
Recall	0.9926	0.9943	0.9880	0.9330	0.8987	0.7737	0.9926	0.9986	0.9991	**0.9992**
F1-Score	0.9925	0.9945	0.9882	0.9330	0.8986	0.7712	0.9925	0.9986	0.9991	**0.9992**
TTTr (seconds)	2227.99	1162.17	923.66	701.17	723.00	719.84	1162.99	1669.48	1655.01	2571.80
TTTe (seconds)	8.57	8.70	7.82	6.13	6.57	7.24	9.74	9.82	10.86	12.55

TABLE VI
COMPARISON OF PERFORMANCE BETWEEN DIFFERENT IDS MODELS FOR *All-Class-Classification* (MULTI-CLASS) OF NETWORK TRAFFIC ON THE HCRL-BUS-TYPE CAN DATASET

Metrics	State-of-the-art Intrusion Detection Systems (SOTA-IDS)						RISK-4-AUTO Models (proposed)			
	LSTMIDS	**GIDS**	**CANintelliIDS**	**DCNN**	**CANTransfer**	**RNN+Heuristics**	**LIN-KAN**	**RES-KAN**	**RES-KAN^{BN}**	**RES-KAN^{II}**
Accuracy	0.9135	0.8934	0.8260	0.8099	0.7623	0.7103	0.9022	0.9166	0.9202	**0.9303**
Precision	0.9010	0.8855	0.8231	0.7941	0.7698	0.7027	0.8948	0.8955	0.9086	**0.9206**
Recall	0.9025	0.8901	0.8216	0.7989	0.7616	0.7024	0.9057	0.8934	0.9098	**0.9247**
F1-Score	0.9051	0.8873	0.8248	0.8009	0.7637	0.7021	0.8829	0.8906	0.9023	**0.9198**
TTTr (seconds)	1422.74	568.21	655.09	696.74	743.88	681.18	512.54	1192.06	1242.14	1892.49
TTTe (seconds)	4.51	4.69	4.13	3.51	3.90	3.83	5.36	5.71	6.01	7.18

TABLE VII
COMPARISON ON CLASSIFYING (BINARY- *Injected*, OR *Normal* MESSAGE) ***DoS-Attack*** FROM HCRL-CHD

Metrics	SOTA-IDS [13]	LIN-KAN	RES-KAN	RES-KAN^{BN}	RES-KAN^{II}
Accuracy	0.9796	0.9979	0.9999	**1.0000**	**1.0000**
Precision	0.9680	0.9979	0.9997	**1.0000**	**1.0000**
Recall	0.9700	0.9978	1.0000	**1.0000**	**1.0000**
F1-Score	0.9690	0.9979	0.9998	**1.0000**	**1.0000**
TTTr (s)	321.02	95.22	242.53	229.46	453.19
TTTe (s)	5.53	4.50	5.66	6.02	6.62



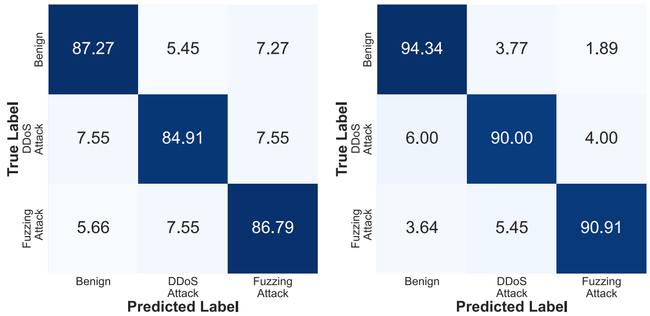
(a) Generative Adversarial Network based IDS (state-of-the-art model)
(b) Dual Residual KAN IDS (best-of-the-proposed model)

Fig. 5. Confusion Matrices for *all-class-classification* of network traffic on the HCRL CAN-FD.

A. Results

Since this work is related to classification, common performance metrics – Accuracy, Precision, Recall, and F1-Score are considered for evaluating the models. The three datasets (as mentioned in Section V-B) are subjected to a 80:10:10 train-test-validation ratio for the experiments. Additionally, the Time-to-Train (TTTr), and Time-to-Test (TTTe) were measured to present the computational overhead of the proposed models, alongside comparing them to the temporal dependencies of the pre-existing models. From a practitioner's point of view, the high training time might seem like a trade-off for achieving more accuracy, but **note that training the model is a one-time process. Once the model has been trained and saved, the same model can be queried any number of times with very little computational overhead (Time-to-Test).**

The following S-O-T-A IDS were considered as baseline (mentioned along with their respective publication venues in chronological order by their publication years), (I) LSTM Autoencoder based IDS (LSTMIDS) [13], (II) GAN based IDS (GIDS) [12], (III) CNN and Attention-based Gated Recurrent Unit (GRU) CAN IDS (CANintelliIDS) [11], (IV) Deep Convolutional Neural Network IDS (DCNN) [3], (V) Transfer Learning based IDS (CANTransfer) [10], and (VI) Heuristic RNN IDS (RNN+Heuristics) [9]. Tables IV, V, and VI present the *all-class* classification (five classes in HCRL-CHD, and



(a) LSTM Autoencoder based IDS (state-of-the-art model)

(b) Dual Residual KAN IDS (RES-KAN^{II}) (best-of-the-proposed model)

Fig. 6. Confusion Matrices for *all-class-classification* of network traffic on the HCRL-B-CAN.

TABLE VIII
COMPARISON ON CLASSIFYING (BINARY- *Injected*, OR *Normal* MESSAGE)
Fuzzy-Attack FROM HCRL-CHD

Metrics	SOTA-IDS [13]	LIN-KAN	RES-KAN	RES-KAN^{BN}	RES-KAN^{II}
Accuracy	0.9806	0.9979	**1.0000**	**1.0000**	**1.0000**
Precision	0.9736	0.9976	**1.0000**	**1.0000**	**1.0000**
Recall	0.9750	0.9981	**1.0000**	**1.0000**	**1.0000**
F1-Score	0.9738	0.9978	**1.0000**	**1.0000**	**1.0000**
TTTr (s)	454.93	147.63	309.81	282.33	539.76
TTTe (s)	5.91	3.58	3.62	4.81	6.05

TABLE IX
COMPARISON ON CLASSIFYING (BINARY- *Injected*, OR *Normal* MESSAGE)
Gear-Spoofing-Attack FROM HCRL-CHD

Metrics	SOTA-IDS [13]	LIN-KAN	RES-KAN	RES-KAN^{BN}	RES-KAN^{II}
Accuracy	0.9783	0.9990	**1.0000**	**1.0000**	**1.0000**
Precision	0.9831	0.9984	**1.0000**	**1.0000**	**1.0000**
Recall	0.9779	0.9985	**1.0000**	**1.0000**	**1.0000**
F1-Score	0.9803	0.9985	**1.0000**	**1.0000**	**1.0000**
TTTr (s)	666.52	98.86	404.13	335.91	711.41
TTTe (s)	6.25	5.92	5.47	6.50	6.42

TABLE X
COMPARISON ON CLASSIFYING (BINARY- *Injected*, OR *Normal* MESSAGE)
RPM-Spoofing-Attack FROM HCRL-CHD

Metrics	SOTA-IDS [13]	LIN-KAN	RES-KAN	RES-KAN^{BN}	RES-KAN^{II}
Accuracy	0.9620	0.9956	0.9982	0.9998	**1.0000**
Precision	0.9658	0.9960	0.9987	0.9999	**1.0000**
Recall	0.9615	0.9953	0.9990	0.9997	**1.0000**
F1-Score	0.9631	0.9956	0.9988	0.9999	**1.0000**
TTTr (s)	623.36	123.45	328.38	320.15	837.66
TTTe (s)	6.07	5.44	4.98	5.73	6.01

TABLE XI
COMPARISON ON CLASSIFYING (BINARY- *Injected*, OR *Normal* MESSAGE)
Flooding-Attack FROM HCRL CAN-FD

Metrics	SOTA-IDS [12]	LIN-KAN	RES-KAN	RES-KAN^{BN}	RES-KAN^{II}
Accuracy	0.9892	0.9992	**1.0000**	**1.0000**	**1.0000**
Precision	0.9901	0.9992	**1.0000**	**1.0000**	**1.0000**
Recall	0.9872	0.9992	**1.0000**	**1.0000**	**1.0000**
F1-Score	0.9884	0.9992	**1.0000**	**1.0000**	**1.0000**
TTTr (s)	300.83	336.58	577.07	320.15	966.78
TTTe (s)	5.62	5.13	5.04	5.41	6.26

three classes each in HCRL-CAN-FD, and HCRL-B-CAN) of the network flows, compared to six state-of-the-art intrusion detection systems.

TABLE XII
COMPARISON ON CLASSIFYING (BINARY- *Injected*, OR *Normal* MESSAGE)
Fuzzy-Attack FROM HCRL CAN-FD

Metrics	SOTA-IDS [12]	LIN-KAN	RES-KAN	RES-KAN^{BN}	RES-KAN^{II}
Accuracy	0.9975	**1.0000**	**1.0000**	**1.0000**	**1.0000**
Precision	0.9975	**1.0000**	**1.0000**	**1.0000**	**1.0000**
Recall	0.9974	**1.0000**	**1.0000**	**1.0000**	**1.0000**
F1-Score	0.9975	**1.0000**	**1.0000**	**1.0000**	**1.0000**
TTTr (s)	316.69	291.81	735.38	769.76	1505.60
TTTe (s)	6.14	6.09	6.61	6.74	7.01

TABLE XIII
COMPARISON ON CLASSIFYING (BINARY- *Injected*, OR *Normal* MESSAGE)
Malfunction-Attack FROM HCRL CAN-FD

Metrics	SOTA-IDS [12]	LIN-KAN	RES-KAN	RES-KAN^{BN}	RES-KAN^{II}
Accuracy	0.9992	**1.0000**	**1.0000**	**1.0000**	**1.0000**
Precision	0.9991	**1.0000**	**1.0000**	**1.0000**	**1.0000**
Recall	0.9992	**1.0000**	**1.0000**	**1.0000**	**1.0000**
F1-Score	0.9992	**1.0000**	**1.0000**	**1.0000**	**1.0000**
TTTr (s)	259.77	229.56	678.04	711.40	1265.91
TTTe (s)	5.23	5.32	6.01	6.36	6.80

TABLE XIV
COMPARISON ON CLASSIFYING (BINARY- *Injected*, OR *Normal* MESSAGE)
DDoS-Attack FROM HCRL-B-CAN

Metrics	SOTA-IDS [13]	LIN-KAN	RES-KAN	RES-KAN^{BN}	RES-KAN^{II}
Accuracy	**1.0000**	0.9972	**1.0000**	**1.0000**	**1.0000**
Precision	**1.0000**	0.9972	**1.0000**	**1.0000**	**1.0000**
Recall	**1.0000**	0.9971	**1.0000**	**1.0000**	**1.0000**
F1-Score	**1.0000**	0.9972	**1.0000**	**1.0000**	**1.0000**
TTTr (s)	209.60	177.67	435.64	448.75	692.15
TTTe (s)	4.89	4.06	4.28	5.00	5.47

TABLE XV
COMPARISON ON CLASSIFYING (BINARY- *Injected*, OR *Normal* MESSAGE)
Fuzzy-Attack FROM HCRL-B-CAN

Metrics	SOTA-IDS [13]	LIN-KAN	RES-KAN	RES-KAN^{BN}	RES-KAN^{II}
Accuracy	0.9985	0.9992	**1.0000**	**1.0000**	**1.0000**
Precision	0.9984	0.9992	**1.0000**	**1.0000**	**1.0000**
Recall	0.9985	0.9992	**1.0000**	**1.0000**	**1.0000**
F1-Score	0.9985	0.9992	**1.0000**	**1.0000**	**1.0000**
TTTr (s)	259.00	239.60	553.04	574.39	1664.85
TTTe (s)	5.01	4.81	4.92	5.38	6.00

The best set of results corresponding to each stratum of *multi-class classification* have been highlighted in **bold** font. Drawing from the experimental observations from the aforementioned tables, for HCRL-CHD, the **best** of the proposed RISK-4-AUTO models outperformed the **best** from the state-of-the-art by 0.7576%; even the **base** variant of the proposed models –RES-KAN outperforms by 0.6568%. Alongside to reprove the overfitting issues, their confusion matrices have also been reported (refer to Fig. 4). Similarly, for HCRL-CAN-FD and HCRL-B-CAN, the **best** of the proposed models outperformed the **best** from the state-of-the-art by 0.4523%, and 1.8398% respectively; and the **base** variant of by 0.3920%, and 0.3393% respectively. The respective confusion matrices for the HCRL-CAN-FD and HCRL-B-CAN have also been reported in Fig. 5, and 6. Apart from *all-class classification*, this research also presents the focused *single-class* attack detection (i.e., whether the attack is performed by *injected*, or *normal message* flow). Corresponding to each of the four malicious flows in HCRL-CHD the *single-class* attack detection results have been presented in Table VII, VIII, IX, and X. Similarly, Tables XI, XII, and XIII presents the

TABLE XVI
IMPLICATIONS OF DIFFERENT COMPUTATIONAL LAYERS FOR THE CLASSIFICATION PROCESS

Layer specific Update (particular to classification)		**Initial Observational Metrics (old configuration)**				**Post-update Observational Metrics (updated configuration)**			
Old Configuration	*New Configuration*	**Accuracy (stdev.)**	**TTTr (sec.)**	**TTTe (sec.)**	**Model Size (kB)**	**Accuracy (stdev.)**	**TTTr (sec.)**	**TTTe (sec.)**	**Model Size (kB)**
`Linear` (×1)	`Linear` (×2)	0.8914 (0.0029)	420.27	5.03	577	0.9192 (0.0035)	478.96	5.21	690
`Linear` (×2)	`Linear` (×3)	0.9192 (0.0035)	478.96	5.21	690	0.9205 (0.0028)	591.29	5.73	800
`Linear` (×2)	`Linear` + `Residual` Block	0.9192 (0.0035)	478.96	5.21	690	0.9627 (0.0024)	600.11	7.39	869
`Linear` + `Residual` Block	`Linear` (×2) + `Residual` Block (×2)	0.9627 (0.0024)	600.11	7.39	869	0.9613 (0.0034)	621.21	7.24	902
`Linear` + `Residual Block`	`Residual Block` (×2)	0.9627 (0.0024)	600.11	7.39	869	0.9807 (0.0049)	810.02	9.15	1051
`Residual` Block (×2)	`Residual` Block (×2)	**Baseline Model**							
`Residual` Block (×2)	`Residual` Block (×3)	0.9807 (0.0049)	810.02	9.15	1051	0.9811 (0.0042)	1004.93	9.47	1544
`KAN-Linear` (×1)	`KAN-Linear` (×2)	0.9883 (0.0014)	620.13	8.46	2435	0.9887 (0.0010)	781.61	8.51	3801
`KAN-Linear` (×2)	`KAN-Linear` + `Residual Block` (×2)	0.9887 (0.0010)	781.61	8.51	3801	0.9966 (0.0013)	1134.82	9.61	3844
`KAN-Linear` + `Residual Block` (×2)	`KAN-Linear` (×2) + `Residual` Block (×2)	0.9966 (0.0013)	1134.82	9.61	3844	0.9976 (0.0008)	2438.17	12.01	96623
`KAN-Linear` (×2) + `Residual` Block (×2)	`KAN-Linear` (×3) + `Residual` Block (×2)	0.9976 (0.0008)	2438.17	12.01	96623	0.9951 (0.0011)	3119.48	13.72	503684

focused *single-class* attack detection metrics for HCRL-CAN-FD (three malicious flows); and Tables XIV, and XV presents the focused attack detection metrics for HCRL-B-CAN (two malicious flows). For each of the tables, one can observe, that they outperform the state-of-the-art models when evaluated single-class by ≈2.5535% on an average, with a deviation of 0.4596.

B. Ablation Study

We performed an ablation study to judge the impact of the different computational layers in the neural networks considered by us. Table XVI presents the layer-wise ablation study with respect to the accuracy (validation), TTTr (sec.), TTTe (sec.) and the model size (kB). Although it is common practice to present the model sizes in terms of the number of parameters, we feel that from an implementation point of view, model size (in kB) would enable the practitioner to make a better choice of the model based on their requirements. Prioritizing on the model accuracy (as a primary decider), a color-code of `green`, `yellow`, and `red` is adopted denoting significant improvement, negligible changes, and degradation respectively.

Further, the collection of four different models allows the practitioner to make an informed choice of the model they might need, based on requirements. If the need of the hour is *achieving more accuracy*, without much constraints on the spatiotemporal requirements of the model, complex variants (Batch-Normalized Residual KAN (RES-KAN ^{BN}) and Dual-Residual-KAN (RES-KAN^{II}) should be considered. But, if it is needed to implement a *robust, lightweight system*, Residual KAN (RES-KAN) can be thought of.

VII. CONCLUSION

We have proposed RISK-4-AUTO, a collection of four deep-learning-based models for the classification of in-vehicle network traffic. The models under RISK-4-AUTO combine the novel Kolmogorov-Arnold Network (enabling trainable activation functions) with the Residual Network architecture. The models were applied on the open source HCRL Datasets, and when evaluated based on their performance compared to other state-of-the-art intrusion detection systems, an **≈1.0163% improvement on an all-class classification**, and **≈2.5535% improvement on focused malicious flow detection** was observed. Additionally, these models can be extended to other in-vehicle networks, e.g., Automotive Ethernet, Serial Peripheral Interface, Safety Systems, etc.

Although the proposed set of RISK-4-AUTO models can work well for the ten different attacks across three datasets, it cannot be generalized for completely out-of-the-box attacks. To the best of our knowledge, except GIDS [12], no other models are generalizable beyond the attacks they have been trained for, and even in GIDS the GAN-based augmentation has been reported to work well on attacks, similar to those on which the models have been trained. Our future research will be directed towards detecting *fabricated message* based attacks.

REFERENCES

[1] M. Kaiser, U. Schaefer, and G. Haaf, “Electronic control unit,” in *Automotive Mechatronics: Automotive Networking, Driving Stability Systems, Electronics*. Wiesbaden, Germany: Springer, 2014, pp. 18–43.

[2] G. M. Smith. “What is can bus (controller area network) and how it compares to other vehicle bus networks.” Accessed: Dec. 2024. [Online]. Available: https://dewesoft.com/blog/what-is-can-bus

[3] H. M. Song, J. Woo, and H. K. Kim, "In-vehicle network intrusion detection using deep convolutional neural network," *Veh. Commun.*, vol. 21, Jan. 2020, Art. no. 100198.
[4] L. B. Othmane, L. Dhulipala, M. Abdelkhalek, N. Multari, and M. Govindarasu, "On the performance of detecting injection of fabricated messages into the can bus," *IEEE Trans. Dependable Secure Comput.*, vol. 19, no. 1, pp. 468–481, Jan./Feb. 2020.
[5] A. Anwar, A. Anwar, L. Moukahal, and M. Zulkernine, "Security assessment of in-vehicle communication protocols," *Veh. Commun.*, vol. 44, Dec. 2023, Art. no. 100639.
[6] F. Arena, G. Pau, and A. Severino, "A review on IEEE 802.11p for intelligent transportation systems," *J. Sens. Actuat. Netw.*, vol. 9, no. 2, p. 22, 2020.
[7] Y. Wang, A. Ahmed, B. Krishnamachari, and K. Psounis, "IEEE 802.11p performance evaluation and protocol enhancement," in *Proc. IEEE Int. Conf. Veh. Electron. Safety*, 2008, pp. 317–322.
[8] A. R. Bharadwaj, M. K. Akhilesh, H. A. Abhiram, and R. Ravish, "Popcorn attack: Intrusion into can remotely through android application of infotainment system," in *Proc. Int. Conf. Knowl. Eng. Commun. Syst. (ICKECS)*, vol. 1, 2024, pp. 1–6.
[9] S. Tariq, S. Lee, H. K. Kim, and S. S. Woo, "Detecting in-vehicle CAN message attacks using heuristics and RNNs," in *Proc. Int. Workshop Inf. Oper. Technol. Secur. Syst.*, 2019, pp. 39–45.
[10] S. Tariq, S. Lee, and S. S. Woo, "CANTransfer: Transfer learning based intrusion detection on a controller area network using convolutional LSTM network," in *Proc. 35th Annu. ACM Symp. Appl. Comput.*, 2020, pp. 1048–1055.
[11] A. R. Javed, S. Ur Rehman, M. U. Khan, M. Alazab, and T. Reddy, "CANintelliiDS: Detecting in-vehicle intrusion attacks on a controller area network using CNN and attention-based GRU," *IEEE Trans. Netw. Sci. Eng.*, vol. 8, no. 2, pp. 1456–1466, Apr.–Jun. 2021.
[12] E. Seo, H. M. Song, and H. K. Kim, "GIDS: GAN based intrusion detection system for in-vehicle network," in *Proc. 16th Annu. Conf. Privacy, Security Trust (PST)*, 2018, pp. 1–6.
[13] J. Ashraf, A. D. Bakhshi, N. Moustafa, H. Khurshid, A. Javed, and A. Beheshti, "Novel deep learning-enabled LSTM autoencoder architecture for discovering anomalous events from intelligent transportation systems," *IEEE Trans. Intell. Transp. Syst.*, vol. 22, no. 7, pp. 4507–4518, Jul. 2021.
[14] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2016, pp. 770–778.
[15] H.-K. Lim, J.-B. Kim, J.-S. Heo, K. Kim, Y.-G. Hong, and Y.-H. Han, "Packet-based network traffic classification using deep learning," in *Proc. Int. Conf. Artif. Intell. Inf. Commun. (ICAIIC)*, 2019, pp. 46–51.
[16] C. Su, Y. Liu, and X. Xie, "Fine-grained traffic classification based on improved residual convolutional network in software defined networks," *IEEE Latin America Trans.*, vol. 21, no. 4, pp. 565–572, Apr. 2023.
[17] Z. Liu et al., "KAN: Kolmogorov-Arnold networks," 2024, *arXiv:2404.19756*.
[18] A. D. M. Ibrahum, Z. Shang, and J.-E. Hong, "How resilient are Kolmogorov–Arnold networks in classification tasks? A robustness investigation," *Appl. Sci.*, vol. 14, no. 22, 2024, Art. no. 10173.
[19] M. Abd Elaziz, I. A. Fares, and A. O. Aseeri, "CKAN: Convolutional Kolmogorov–Arnold networks model for intrusion detection in IoT environment," *IEEE Access*, vol. 12, pp. 134837–134851, 2024.
[20] H. K. Kim et al. "HCRL-car hacking dataset." 2020. Accessed: Dec. 2024. [Online]. Available: https://ocslab.hksecurity.net/Datasets/car-hacking-dataset
[21] H. K. Kim. "HCRL-can with flexible data rate dataset." 2022. Accessed: Dec. 2024. [Online]. Available: https://ocslab.hksecurity.net/Datasets/can-fd-intrusion-dataset
[22] H. Kim. "HCRL-B-can dataset." 2022. Accessed: Dec. 2024. [Online]. Available: https://ocslab.hksecurity.net/Datasets/b-can-intrusion-dataset,
[23] L. Yang, A. Moubayed, and A. Shami, "MTH-IDS: A multitiered hybrid intrusion detection system for Internet of Vehicles," *IEEE Internet Things J.*, vol. 9, no. 1, pp. 616–632, Jan. 2022.
[24] P. F. De Araujo-Filho, A. J. Pinheiro, G. Kaddoum, D. R. Campelo, and F. L. Soares, "An efficient intrusion prevention system for can: Hindering cyber-attacks with a low-cost platform," *IEEE Access*, vol. 9, pp. 166855–166869, 2021.
[25] T. Shapira and Y. Shavitt, "FlowPic: A generic representation for encrypted traffic classification and applications identification," *IEEE Trans. Netw. Service Manag.*, vol. 18, no. 2, pp. 1218–1232, Jun. 2021.
[26] T. Shapira, "FlowPic: Encrypted internet traffic classification is as easy as image recognition," in *Proc. IEEE Conf. Comput. Commun. Workshops (INFOCOM WKSHPS)*, 2019, pp. 680–687.
[27] E. Horowicz, T. Shapira, and Y. Shavitt, "A few shots traffic classification with mini-flowpic augmentations," in *Proc. 22nd ACM Internet Meas. Conf.*, 2022, pp. 647–654.
[28] M. Z. Alom et al., "The history began from AlexNet: A comprehensive survey on deep learning approaches," 2018, *arXiv:1803.01164*.
[29] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet classification with deep convolutional neural networks," *Commun. ACM*, vol. 60, no. 6, pp. 84–90, 2017.
[30] K. He, R. Girshick, and P. Dollár, "Rethinking ImageNet pre-training," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, 2019, pp. 4918–4927.
[31] O. Russakovsky et al., "ImageNet large scale visual recognition challenge," *Int. J. Comput. Vis.*, vol. 115, pp. 211–252, Dec. 2015.
[32] S. Hochreiter, "The vanishing gradient problem during learning recurrent neural nets and problem solutions," *Int. J. Uncertainty, Fuzziness Knowl. Based Syst.*, vol. 6, no. 2, pp. 107–116, 1998.
[33] G. Philipp, D. Song, and J. G. Carbonell, "The exploding gradient problem demystified-definition, prevalence, impact, origin, tradeoffs, and solutions," 2017, *arXiv:1712.05577*.
[34] A. Dutta et al. "RISK-4-AUTO codebase." Accessed: Dec. 2024. https://github.com/Anurag-Dutta/RISK-4-Auto,

Anurag Dutta (Graduate Student Member, IEEE) received the bachelor's degree in computer science and engineering from the Government College of Engineering and Textile Technology, Serampore, India. He is currently pursuing the M.S. degree with the Department of Computer Science and Engineering, IIT Kharagpur. His research interests include time-series forecasting, computer vision, image processing, and the applications of artificial intelligence in digital forensics.

Sangita Roy (Member, IEEE) received the B.Tech. degree from the West Bengal University of Technology and the M.Tech. degree from the Kalinga Institute of Industrial Technology, Bhubaneswar, and the Ph.D. degree from IIT Patna. She is currently an Assistant Professor with the Department of Computer Science and Engineering, Thapar Institute of Engineering and Technology, Patiala, India. Her research interests include computer network, network security, image processing, and deep learning.

Rajat Subhra Chakraborty (Senior Member, IEEE) is a Professor with the Department of Computer Science and Engineering, IIT Kharagpur. His area of research is hardware security, VLSI design, and digital content protection through watermarking. He is a Senior Member of ACM.